

EXERCISES - PART 1

1. We have a class Company that stores two types of employees: HourlyEmployee and MonthlyEmployee. Out of the next two versions of class Company which one would you like to use? Explain your decision.

It depends on what we want to do

```
class CompanyA {  
    private ArrayList<HourlyEmployee> hourlyEmployees;  
    private ArrayList<MonthlyEmployee> monthlyEmployees;  
  
    public void add(Employee e) { . . .  
    public String toString() { // return all the stored employees as a String  
}
```

```
class CompanyB {  
    private ArrayList<Employee> employees;  
  
    public void add(Employee e) { . . .  
    public String toString() { // return all the stored employees as a String  
}
```

2. a) Which rule/principle/pattern is violated in the next fragment of code? Refactor to get rid of the detected problem.

Law of Demeter is violated

```
class BuildingTaxCalculator {  
    private BuildingTaxTable table;  
    public BuildingTaxCalculator(BuildingTaxTable table) { this.table = table; }  
  
    public int compute(Person person, Building building) {  
        return building.getSquareFeetNumber() *  
            table.getTax(person.getAddress());  
    }  
}
```

b) What is wrong in the next source code?

```
class Engine { . . . }  
class Car {  
    private Engine e;  
    public Car () {  
        this.e = new Engine();  
    }  
}
```

Seeing that the only purpose of person is to get the address, in order not to violate the law of Demeter, our function could just receive the address directly as a parameter

DIP is violated because we depend on a concrete thing instead of an abstraction

+ Liskov because higher modules depend on lower ones

3. Name the design principle that is violated in the next source code. Modify the code in order to remove the problem.

```
interface Toy {  
    // postcondition - returns a String that is not null  
    public String play();  
}  
  
class Car implements Toy {  
    public String play() {  
        return "Playing with the car";  
    }  
}
```

Liskov Substitution Principle is violated because we return something that we don't expect.

```
class ElectricCar implements Toy {
    private boolean isTurnedOn = false;

    public void turnOn() { isTurnedOn = true; }

    public String play() {
        if(isTurnedOn)
            return "Playing with an electric car";
        return null;
    }
}
```

To fix it instead of returning null, we'll just return a string

```
class ToyTest{
    public static void main(String[] args){
        Toy t = new Car();
        t.play();
        Toy o = new ElectricCar();
        o.play();
    }
}
```

4. Given the next class

ArrayList<Employee> employees

```
class Employee {
    private String firstName, lastName;
    public Employee(String firstName, lastName) {
        // . . .
    }
}
```

Instantiate an object that contains Employee objects and print the kept employees. Inside the instantiated object we can store more than once the same employee. If needed, add new services inside the given class.

5. We have a class Company that stores two types of employees: HourlyEmployee and MonthlyEmployee. Out of the next two versions of class Company which one would you like to maintain? Explain your decision.

```
class CompanyA {
    private ArrayList<HourlyEmployee> hourlyEmployees;
    private ArrayList<MonthlyEmployee> monthlyEmployees;
```

```
    public void add(Employee e) { . . .
    public String toString() { // return all the stored employees as a String
    }
```

```
class CompanyB {
    private ArrayList<Employee> hourlyEmployees;
```

```
    public void add(Employee e) { . . .
    public String toString() { // return all the stored employees as a String
    }
```

this one would be easier to maintain because it uses polymorphism and adding a new employee requires zero change

6. a) Is Demeter's law violated in the next fragment of code? Explain the reasons behind your answer.

```
class Colada {
    private Blender myBlender;
    private ArrayList<Ingredient> ingredients;
```

→ Law of Demeter isn't violated because we only call methods of the classes of the attributes of the class

1. SRP is violated because higher level modules rely on lower-level modules instead of an abstraction

2. OCP if we were to change the ingredients we would also have to change the blender and also the Coloda

```
public Colada(ArrayList<Ingredient> ingredients) {  
    myBlender = new Blender();  
    this.ingredients = ingredients;  
}  
  
public void mix() {  
    myBlender.addIngredients(ingredients);  
}}
```

b) It seems the programmer who wrote the code above does not know an important SOLID principle – name the principle the programmer forgot to comply with and refactor the code in order to remove the existing negative consequences.

7. Does the Service interface comply/not comply with the Interface Segregation Principle?

```
interface Service {  
    public void firstService();  
    public int computeService();  
    public int multiplyService();  
}
```

ISP refers to clients. Without knowing what the clients needs or wants to do, we don't know.

8. Given the next class

```
class Employee {  
    private String firstName, lastName;  
    public Employee(String firstName, lastName) { // . . . }}
```

Instantiate an object that contains Employee objects and print the kept employees. Define the needed services that guarantee an existing person is stored only once. If needed, add new services inside the given class.

Set<Employee> employees

9. Based on the next Vehicle interface

```
interface Vehicle {  
    public void enableHeatInChair();  
    public void disableHeatInChair();  
    public void startEngine();  
    public void accelerate();  
}
```

ISP is violated because a client shouldn't have to use something they don't need. To fix that we make multiple more specific interfaces

SRP
we intend to define various types of vehicles and some of them provide all the presented services and some of them do not (e.g. implement the service that does not exist with a No Operation – an empty method). Some clients of this interface use all the provided services and some do not. Identify at least two problems related to this implementation, name the principles this approach does not comply with and show the necessarily changes that have to be performed in order to get rid of the problems.

also violated

10. a) Does the following class violate the Single Responsibility Principle? If so, refactor it!

```
class Student {  
    . . .  
    public void saveStudent() { . . . //save into a file  
    public Double computeGrade() { . . .  
    public String getStudentProfile() { . . . //return firstName + lastName +  
    grade  
}
```

yes because this class does a lot of very different jobs and has multiple reasons to change

b) Does the Student class comply/not comply with the Interface Segregation Principle?

NO

11. We have a class University that stores two types of persons: Teachers and Students. Out of the next two versions of class University which one would you like to use? Explain your decision.

```
class UniversityA {
    private Collection<Teacher> teachers;
    private Collection<Student> students;

    public void add(Teacher t) { . . .
    public void add(Student s) { . . .

    public String toString() { // return the stored teachers + students as a String
    }
}

class UniversityB {
    private Collection<Person> persons; //both Teacher and Student extend Person

    public void add(Person e) { . . .
    public String toString() { // return the stored teachers + students as a String
    }
}
```

this one would be much easier to maintain. We could extend this class and have 2 subclasses Student and Teachers

12. Based on the next Vehicle interface

```
interface Vehicle {
    public void startEngine();
    public void accelerate();
    public void stopRadio();
    public void ejectCD();
}
```

SAME AS 9

, we intend to define various types of vehicles and some of them provide all the presented services and some of them do not (e.g. implement the service that does not exist with a No Operation – an empty method). Some clients of this interface use all the provided services and some do not. Identify at least two problems related to this implementation, name the principles this approach does not comply with and show the necessarily changes that have to be performed in order to get rid of the problems.

ISP refers to the fact that a client shouldn't be forced to use something they don't need. Also, there is no limit as to how many interfaces a class can implement.

13. Comment the next statements:

a) The Interface Segregation Principle refers to the fact that each class should implement a small number of interfaces, usually not more than four. *NO*

b) We should always create classes that do not have accessor methods (getters and setters).

c) If a class violates the Single Responsibility Principle it should provide more than two services. *should have multiple reasons to change*

d) Following Demeter's Law means accessing data from a restricted set of objects.

Yes. We can't access everything. We don't talk to everybody, only friends.

14. a) What is a God Class? *a class that has a lot of responsibilities*

b) Which design principle you think is always violated by God Classes? Why?

c) How do you think you can capture the God Classes inside a system by looking only at its class diagram?

SRP is violated by a god class because of the multiple responsibilities

15. Considering a Base class, a Derived class that is derived from Base and the next methods, you have to specify whether or not the Liskov substitution principle is violated. Justify your answer by relating the pre and post conditions of the Derived class to the ones from the Base class.

Base Class

Precondition

Postcondition

Derived Class

→ can be equal or stricter

→ can be equal or stronger

→ returns something unexpected

Base: `int service(int number) //PRECOND:  $x > 50$  //POSTCOND: returns 1 OR 2`

Derived: `int service(int number) //PRECOND:  $x > 30$  //POSTCOND: returns 1 OR 2 OR 3`

16. Define the Interface Segregation Principle.

The Interface Segregation Principle refers to the fact that a client shouldn't be forced to use things he doesn't need. $\Rightarrow$ multiple more specific interfaces are better

17. Comment the next statements:

- a) We can detect the classes that violates The Interface Segregation Principle by looking at the interfaces that are implemented by each class. *NOT enough*
- b) If we add getters to a class then using the returned objects means always violating Demeter's Law. *False. Using multiple getters can violate Demeter, but that doesn't mean that*
- c) We should always create classes that do not have accessor methods (getters and setters). *mean that*
- d) It is always better to have getters and setters *it always does*
in order not to expose internal state unnecessarily

18. a) What is a hook method?

- b) Define Fragility and Immobility.
- c) Define Rigidity and Viscosity.

→ A hook method is a method that is a template so that methods in subclasses can override it and add customized behavior without changing the overall structure

Fragility = changing even a little can mess everything up

Immobility = you can't extract/reuse it

Rigidity = very hard to change it. Take a lot of time and resources

Viscosity = the resistance to change

SOLID PRINCIPLES

- S → single responsibility principle
- O → open-closed principle
- L → Liskov Substitution principle
- I → Interface Segregation principle
- D → Dependency inversion principle

Single Responsibility principle

a class, module or function should have only one responsibility

= one reason to change

Open-Closed principle

→ a class should be open for extension but closed for modification

= we need to be able to add new functionality without changing the existing code

Liskov Substitution principle

→ subtypes must be substitutable for their base types

= objects of a subclass should behave correctly when used in place of their parent class

ex:

If the class Bird has fly(), a penguin subclass shouldn't override it to throw an error. Instead there should be a FlyingBird subclass

Interface Segregation Principle

→ a client should not depend on interfaces it does not use

⇒ break large generalized interfaces into smaller more specific ones so that classes don't need to implement unnecessary methods

Dependency Inversion Principle

→ high-level modules should not depend on low-level modules. Both should depend on abstractions

= code should be dependent on interfaces and abstract classes, not on concrete implementations

The Contract of a method

method contract = defines the expectations and guarantees of a method

Preconditions: what must be true before a method is called

Postconditions: what will be true after the method finishes

Invariants: what remains consistently true during execution

Side effects: any changes beyond the return value